上下文动态注入方案

目录

1. 概述：上下文注入的分层架构
2. 第一层：System Reminder 动态注入
   - 核心机制
   - 注入场景
   - 注入内容分类
   - Beta Tracing 中的 System Reminder 收集
   - 包裹 `<system-reminder>` 在的消息类型
3. 第二层：CLAUDE.md / 记忆文件注入
   - 文件发现层级
   - 关键函数
   - @ 指令注入
   - HTML 注释 stripping
   - MEMORY.md 入口截断
   - 条件化规则（Conditional Rules）
   - 与 `context.ts` 的集成
   - 缓存清除机制
4. 第三层：Auto Memory 系统（自动记忆）
   - 概述
   - 记忆类型分类
   - 持久化结构
   - 记忆文件 Frontmatter 格式
   - 与 System Prompt 的集成
   - 行为约束指导
   - 过期记忆验证指导
5. 第四层：Session Memory（会话记忆）
   - 概述
   - 触发机制
   - 触发阈值
   - 隔离机制
   - 与 Compaction 的集成
6. 第五层：ExtractMemories 提取记忆服务
   - 概述
   - 触发机制
   - mutual exclusion（互斥机制）
   - 工具权限控制
   - 提取节流
   - trailing run（尾随运行）
7. 第六层：Hooks 系统（生命周期注入）

- 概述
- Hook 类型（按触发时机）
- InstructionsLoaded Hook（条件化注入）
- Hook 输出注入
- asyncRewake 机制（后台 Hook 唤醒）
- 工作目录信任检查

8. 第七层：System Prompt 分段与动态边界
   - SYSTEM_PROMPT_DYNAMIC_BOUNDARY
   - 缓存断裂注入
   - 缓存断裂检测
9. 第八层：条件化规则注入（Conditional Rules）
   - 嵌套记忆加载机制
   - Glob 模式匹配
   - 前置路径解析
10. Post-Sampling Hook（采样后注入）
    - 核心机制
    - 使用场景
11. appendSystemPrompt 与 overrideSystemPrompt
    - 两层额外注入
    - 优先级体系
    - Cowork 额外指导注入
12. Coordinator 与 Agent Prompt 优先级
    - Coordinator 模式
    - Agent 定义注入
13. Compact（上下文窗口压缩）
    - 压缩触发时机
    - 压缩过程中的记忆处理
    - compact boundary 标记
14. 架构总结与关键设计模式
    - 分层职责一览
    - 关键设计模式

---

1. 概述：上下文注入的分层架构

Claude Code 的上下文动态注入并非单一机制，而是由 **13 层独立子系统** 构成，各层职责清晰、边界分明，通过统一的消息构建管道最终汇聚到 API 调用中。整体架构可概括为：

```
API 调用（query()）
  ↑
  ┌─ systemPrompt（来自 prompts.ts / systemPromptSections.ts）
  ├─ userContext（来自 context.ts: getUserContext）
  ├─ systemContext（来自 context.ts: getSystemContext）
  ├─ messages[]（用户消息 + 系统附件消息）
  └─ 附件（attachments: skill, agent, mcp, tool delta 等）
```

2. 第一层：System Reminder 动态注入

2.1 核心机制

System Reminder 是 Claude Code 最基础的上下文注入手段。通过将文本内容包裹在 `<system-reminder>...</system-reminder>` XML 标签中，**将系统级信息注入到用户消息（user message）内部**，使其进入模型的注意力上下文中。

核心函数： `src/utils/messages.ts`

```
// 入口函数
export function wrapInSystemReminder(content: string): string {
  return `<system-reminder>\n${content}\n</system-reminder>`
}

// 批量包装
export function wrapMessagesInSystemReminder(
  messages: Message[],
): Message[]
```

2.2 注入场景

所有通过 `buildSystemInitMessage()` 构建的 `system/init` 类型消息，以及通过各种附件构建函数生成的附件消息，都会被自动包装进 `<system-reminder>` 标签中：

```
// src/utils/messages.ts 附件包装示例
return wrapMessagesInSystemReminder([
  createAttachmentMessage({
    type: 'system_local_command',
    content: `${formatShellPrefixCommand(command, workingDirectory)} ${input}`,
  }),
])
```

2.3 注入内容分类

系统提示文本本身（`prompts.ts`）也包含了对 System Reminder 的说明：

```
function getSystemRemindersSection(): string {
  return `- Tool results and user messages may include <system-reminder> tags.
    <system-reminder> tags contain useful information and reminders. They are
    automatically added by the system, and bear no direct relation to the specific
    tool results or user messages in which they appear.
  - The conversation has unlimited context through automatic summarization.`
}
```

2.4 Beta Tracing 中的 System Reminder 收集

在 `src/utils/telemetry/betaSessionTracing.ts` 中，系统会提取所有消息中的 system-reminder 内容用于遥测：

```
function extractSystemReminderContent(text: string): string | null {
  const match = text.match(/<system-reminder>([\s\S]*?)<\/system-reminder>/)
```

```
  return match ? match[1] : null
}
```

2.5 包裹 `<system-reminder>` 在的消息类型

所有通过 normalizeAttachmentForAPI() 处理的消息，经过 wrapMessagesInSystemReminder() 包裹后，会以两种形式携带 `<system-reminder>` 标签：

- 字符串内容：整体被包裹成 `<system-reminder>\n{content}\n</system-reminder>`
- 数组内容：只包裹 type: 'text' 的 block，保留 type: 'tool_result' 等其他 block 类型不变

真正会被包裹的内容（附件类）：

类型	内容说明
plan_mode / plan_mode_reentry / plan_mode_exit	计划模式工作流指令
auto_mode / auto_mode_exit	自动模式指令
edited_text_file	文件变更通知
compact_file_reference	摘要后旧文件引用
pdf_reference	大 PDF 轻量引用
selected_lines_in_ide	IDE 选中行
opened_file_in_ide	IDE 打开文件通知
plan_file_reference	计划文件内容
invoked_skills	已调用技能内容
todo_reminder	TodoWrite 提醒
task_reminder	TaskCreate/TaskUpdate 提醒
nested_memory / relevant_memories	记忆内容
skill_listing / skill_discovery	技能列表/发现
queued_command	本地排队命令
output_style	输出风格通知
diagnostics	诊断问题
mcp_resource	MCP 资源内容
agent_mention	Agent 提及
task_status	后台任务状态
async_hook_response	Hook 异步响应
token_usage / budget_usd / output_token_usage	限额统计
hook_blocking_error / hook_success / hook_additional_context / hook_stopped_continuation	Hook 事件
compaction_reminder / context_efficiency	自动压缩提示
date_change	日期变更
ultrathink_effort	推理 effort 级别
deferred_tools_delta / agent_listing_delta / mcp_instructions_delta	增量更新通知
companion_intro	Companion 介绍
verify_plan_reminder	VerifyPlanExecution 调用提醒
critical_system_reminder	任意关键提醒
teammate_mailbox / team_context	Swarm 多智能体消息

设计理念：这些条目都是"模型需要据此行动，但不能直接告知用户"的信息。

具体分几类：

直接改变模型行为的指令

plan_mode、auto_mode、verify_plan_reminder、ultrathink_effort —— 用户切换了某种工作模式，系统需要通知模型"你现在该怎么做事"

用户已知的系统状态变更

edited_text_file、selected_lines_in_ide、opened_file_in_ide、date_change —— 用户自己改的文件、自己选的行，模型需要知道这些发生了，但不应在回复里说"我注意到你改了文件"。

工具无法主动获取的背景知识

invoked_skills、relevant_memories、nested_memory、mcp_resource —— 用户没有用工具请求这些信息，是系统主动注入的上下文

系统内部状态/警告

diagnostics、compaction_reminder、context_efficiency、token_usage、budget_usd —— 完全内部化的系统信号，模型即使知道也不应告知用户，更不应据此向用户解释。

Hook 事件流

hook_blocking_error、hook_success、hook_additional_context 等 —— Hook 运行结果的反馈，供模型决策用

---

3. 第二层：CLAUDE.md / 记忆文件注入

3.1 文件发现层级

`src/utils/claudemd.ts` 实现了完整的 CLAUDE.md 文件发现和注入管道。文件按以下优先级加载（低优先级先加载，高优先级后加载，模型对后加载的内容更关注）：

```
加载顺序（先低后高）：
1. Managed（全局管理级，/etc/claude-code/CLAUDE.md）
2. User（用户全局级，~/.claude/CLAUDE.md）
3. Project（项目级，向上遍历目录找到的 CLAUDE.md / .claude/CLAUDE.md / .claude/rules/*.md）
4. Local（本地级，CLAUDE.local.md）

最终合并为一个字符串，以 "Contents of <path>:" 形式逐段拼接
```

3.2 关键函数

```
// 主入口：发现并加载所有记忆文件
export const getMemoryFiles = memoize(
  async (forceIncludeExternal: boolean = false): Promise<MemoryFileInfo[]>
)

// 将文件列表转换为系统提示字符串
export const getClaudeMds = (memoryFiles: MemoryFileInfo[]): string => {
  // 对每种类型添加描述后缀，格式化为：
  // "Contents of /path/to/CLAUDE.md (project instructions, checked into the
codebase):\n\n<content>"
}
```

3.3 @ 指令注入

记忆文件支持通过 `@path` 语法引入其他文件：

```
// @include 支持的路径格式
// @path, @./relative/path, @~/home/path, @/absolute/path
// 仅在纯文本节点中展开，代码块/HTML注释内不展开
// 最大递归深度：5 层
```

PS：记忆文件中的 @path 引用会在加载时被展开，把目标文件内容注入进来

3.4 HTML 注释 stripping

在解析记忆文件内容时，block-level HTML 注释（`<!-- ... -->`）会被自动剥离：

```
// src/utils/claudemd.ts
export function stripHtmlComments(content: string): { content: string; stripped: boolean }
```

PS：

- 作者在 CLAUDE.md 里手动写的 ，本质上是人类-only 的标注
- 这部分内容对应着作者在 CLAUDE.md 中写入但不希望 AI 看到的笔记，本质上是**对人类可见、对 AI 隐藏的标注**

典型场景：

```
1. 内部 TODO / 待办标注
<!-- TODO: 这条规则等 v2 发布后要重写 -->
<!-- FIXME: 临时绕过，正式环境请删除此规则 -->
请严格遵守以下 API 规范...

2. 草稿内容（还未定稿）
<!-- 下面的规范还在讨论中，暂不生效 -->
## 实验性功能处理方式
正在和团队确认中...

3. 仅部分人需要看到的说明
<!-- 给维护者的提示：这个目录结构会在下个版本重构 -->
## 部署配置

4. 多人协作时的上下文注释
<!-- 2024-06 经安全 review 后添加此规则 -->
禁止直接拼接用户输入到 SQL。

<!-- legal-team-2024-07: 此例外条款由法务要求添加 -->

5. 条件性注释（debug / 开发专用）
<!-- DEBUG: 本地开发时忽略证书校验 -->
连接外部服务时请验证证书...
```

设计意图

作者希望在 CLAUDE.md 中保留这些人类协作痕迹（review 记录、TODO、draft 内容），但不希望它们进入 AI 上下文——因为：

- TODO/草稿会让 AI 产生困惑
- 内部分析注释会污染上下文
- 临时 workaround 不应影响 AI 的判断

剥离逻辑的设计是：**注释如果写成块级形式（独占一行）就默认是给人类的标注**；如果写在行内（Use method），说明作者在正文中引用注释语法，那才是有意义的文本，不剥离

3.5 MEMORY.md 入口截断

`MEMORY.md` 入口文件有双重限制：

```
// src/memdir/memdir.ts
export const MAX_ENTRYPOINT_LINES = 200   // 最多 200 行
export const MAX_ENTRYPOINT_BYTES = 25_000 // 最多 25KB

// 截断时附加警告：
// "> WARNING: MEMORY.md is {reason}. Only part of it was loaded..."
```

PS：MEMORY.md 是记忆的索引，只存指针，格式为 `- [标题](文件名.md) — 简短描述`

3.6 条件化规则（Conditional Rules）

文件可以通过 frontmatter `paths` 字段限制适用范围：

```
---
paths:
  - src/**/*.ts
  - src/**/*.tsx
---
Your instructions here...
```

```
// src/utils/claudemd.ts
export async function processConditionedMdRules(
  targetPath: string,
  rulesDir: string,
  type: MemoryType,
  processedPaths: Set<string>,
  includeExternal: boolean,
): Promise<MemoryFileInfo[]>
// 使用 ignore 库做 glob 模式匹配
```

系统通过 `glob` 模式匹配当前工作目录路径，仅在匹配时注入该记忆。

PS：
典型的 **上下文感知** 或者说 **动态提示词注入** 设计，简单来说：

- 普通记忆是"全局生效"的——只要加载了，就一直存在于上下文里
- 条件规则允许记忆文件声明：**只有当我编辑特定路径的文件时，这条记忆才有用，从而只在匹配时注入**

3.7 与 `context.ts` 的集成

`src/context.ts` 中的 `getUserContext()` 调用 `getClaudeMds()`：

```
// src/context.ts
export const getUserContext = memoize(async (): Promise<{ [k: string]: string }> => {
  const claudeMd = getClaudeMds(filterInjectedMemoryFiles(await getMemoryFiles()))
  setCachedClaudeMdContent(claudeMd || null) // 缓存供其他模块使用
  return {
    ...(claudeMd && { claudeMd }),
    currentDate: `Today's date is ${getLocalISODate()}.`,
  }
})
```

3.8 缓存清除机制

```
// 重置缓存（触发完整重新加载）
export function resetGetMemoryFilesCache(reason: InstructionsLoadReason): void

// 仅清除缓存（不触发 hook，用于纯正确性目的）
export function clearMemoryFileCaches(): void

// loadReason 传递：'session_start' | 'compact' | 'include'
let nextEagerLoadReason: InstructionsLoadReason = 'session_start'
```

PS：这两个函数服务于**hook 通知 + 缓存失效联动**，和动态注入的关系在于：**让 hook 的调用方知道 memory 重新加载的原因**，从而决定如何响应（比如压缩后需要重新构建上下文，而 worktree 切换只需要静默刷新缓存）

---

4. 第三层：Auto Memory 系统（自动记忆）

PS：详见 “记忆系统” 相关文档

4.1 概述

Auto Memory 是一个完整的文件型持久记忆系统，通过 `loadMemoryPrompt()` 将引导性提示注入到 System Prompt 中。核心文件：`src/memdir/memdir.ts`。

4.2 记忆类型分类

系统强制将记忆分为 4 种类型（`src/memdir/memoryTypes.ts`）：

```
export const MEMORY_TYPES = ['user', 'feedback', 'project', 'reference'] as const
```

每种类型都有明确的 `when_to_save`、`how_to_use`、`body_structure` 规范。

4.3 持久化结构

```
~/.claude/projects/<slug>/memory/
  MEMORY.md          # 入口索引（每条记录一行链接）
  user_role.md       # 主题文件（frontmatter + 内容）
  feedback_*.md
  project_*.md
  reference_*.md
```

4.4 记忆文件 Frontmatter 格式

```
---
name: {{memory name}}
description: {{one-line description}}
type: {{user, feedback, project, reference}}
---
{{memory content}}
```

4.5 与 System Prompt 的集成

```
// src/memdir/memdir.ts
export async function loadMemoryPrompt(): Promise<string | null> {
  // 1. KAIROS 每日日志模式（append-only）
  if (feature('KAIROS') && autoEnabled && getKairosActive()) {
    return buildAssistantDailyLogPrompt(skipIndex)
  }
  // 2. TEAMMEM 团队记忆模式
  if (feature('TEAMMEM') && teamMemPaths!.isTeamMemoryEnabled()) {
    return teamMemPrompts!.buildCombinedMemoryPrompt(extraGuidelines, skipIndex)
  }
  // 3. 标准 auto memory
  if (autoEnabled) {
    return buildMemoryLines('auto memory', autoDir, extraGuidelines, skipIndex).join('\n')
  }
  return null
}
```

4.6 行为约束指导

Auto Memory 注入的 System Prompt 包含详细的"不应该保存"列表：

```
// src/memdir/memoryTypes.ts
export const WHAT_NOT_TO_SAVE_SECTION: readonly string[] = [
  '## What NOT to save in memory',
  '- Code patterns, conventions, architecture, file paths, or project structure...',
  '- Git history, recent changes, or who-changed-what...',
  '- Debugging solutions or fix recipes...',
  '- Anything already documented in CLAUDE.md files.',
  '- Ephemeral task details...',
  // 重要：即使用户要求保存，也要过滤
  'These exclusions apply even when the user explicitly asks you to save.'
]
```

4.7 过期记忆验证指导

```
// src/memdir/memoryTypes.ts
export const TRUSTING_RECALL_SECTION: readonly string[] = [
  '## Before recommending from memory',
  'A memory that names a specific function, file, or flag is a claim that it existed...',
  '- If the memory names a file path: check the file exists.',
  '- If the memory names a function or flag: grep for it.',
  '"The memory says X exists" is not the same as "X exists now."'
]
```

---

5. 第四层：Session Memory（会话记忆）

PS：详见 “上下文管理” 相关文档

5.1 概述

Session Memory 是一种**会话级**的记忆机制，在对话运行期间自动维护一个 Markdown 文件，记录当前会话的关键信息。核心文件：`src/services/SessionMemory/sessionMemory.ts`。

5.2 触发机制

Session Memory 通过 **Post-Sampling Hook** 触发（详见第九层），在模型采样完成后自动运行：

```
// src/services/SessionMemory/sessionMemory.ts
export async function extractSessionMemory(context: REPLHookContext): Promise<void> {
  // 仅在主 REPL 线程运行
  if (querySource !== 'repl_main_thread') return
  // 通过 fork subagent 执行，不阻塞主对话
  await runForkedAgent({ querySource: 'session_memory', ... })
}
```

5.3 触发阈值

```
// 初始化阈值（对话进行到什么程度开始第一次提取）
minimumMessageTokensToInit: 1500  // token 数量

// 更新阈值（每次提取后，下次提取的最小 token 增长量）
minimumTokensBetweenUpdate: 3000

// 工具调用阈值（距离上次提取，经过多少次工具调用）
toolCallsBetweenUpdates: 50
```

5.4 隔离机制

Session Memory 使用 `runForkedAgent` 创建隔离上下文，防止子进程污染父进程状态：

```
// 隔离 readFileState
overrides: { readFileState: setupContext.readFileState }
// 限制工具：只允许 Edit 特定 memory 文件
canUseTool: createMemoryFileCanUseTool(memoryPath)
```

5.5 与 Compaction 的集成

```
// src/services/compact/sessionMemoryCompact.ts
// 当上下文压缩发生时，session memory 内容也被纳入摘要过程
```

---

6. 第五层：ExtractMemories 提取记忆服务

PS：详见 “记忆系统” 相关文档

6.1 概述

ExtractMemories 服务在每次对话轮次结束后（stopHooks），自动从对话历史中提取值得持久化的信息，写入 Auto Memory 目录。核心文件：`src/services/extractMemories/extractMemories.ts`。

6.2 触发机制

```
// 在 stopHooks.ts 中通过 handleStopHooks 调用
export async function executeExtractMemories(
  context: REPLHookContext,
```

```
  appendSystemMessage?: AppendSystemMessageFn,
): Promise<void>
```

6.3 mutual exclusion（互斥机制）

ExtractMemories 与主 Agent 之间的互斥策略：

```
// 如果主 Agent 刚刚自己写了记忆文件，跳过这次提取
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  return  // 跳过，避免重复
}
```

6.4 工具权限控制

```
// src/services/extractMemories/extractMemories.ts
export function createAutoMemCanUseTool(memoryDir: string): CanUseToolFn {
  // 允许：Read / Grep / Glob（无限制）
  // 允许：只读 Bash 命令（ls, find, grep, cat, stat...）
  // 允许：Edit / Write 仅针对 auto-memory 目录内文件
  // 拒绝：其他所有工具
}
```

6.5 提取节流

```
// 通过 tengu_bramble_lintel GrowthBook 配置控制
// 默认每 1 轮执行一次提取
turnsSinceLastExtraction < getFeatureValue_CACHED_MAY_BE_STALE('tengu_bramble_lintel', 1)
```

6.6 trailing run（尾随运行）

如果提取过程中新消息到达，将 stash 最新上下文，提取完成后立即运行一次尾随提取：

```
if (inProgress) {
  pendingContext = { context, appendSystemMessage }
  // 当前运行结束后执行 trailing run
}
```

---

7. 第六层：Hooks 系统（生命周期注入）

PS：详见 “Hook系统实现” 相关文档

7.1 概述

Hooks 是用户通过 `.claude/settings.json` 配置的 shell 命令或函数回调，在 Claude Code 生命周期中的特定事件点触发，执行后将结果（JSON 输出）注入回对话上下文。核心文件：`src/utils/hooks.ts`。

7.2 Hook 类型（按触发时机）

```
type HookEvent =
  | 'PreToolUse'           // 工具调用前
  | 'PostToolUse'          // 工具调用后
  | 'PostToolUseFailure'   // 工具调用失败后
```

```
  | 'PreCompact'            // 压缩前
  | 'PostCompact'           // 压缩后
  | 'Stop'                  // 对话停止时
  | 'StopFailure'           // 对话异常停止时
  | 'PreExecute'            // 用户提交前
  | 'UserPromptSubmit'      // 用户提交时
  | 'PermissionsChange'     // 权限变更时
  | 'SessionStart'          // 会话启动时
  | 'SessionEnd'            // 会话结束时
  | 'SubagentStart'         // 子代理启动时
  | 'SubagentStop'          // 子代理结束时
  | 'TaskCreated'           // 任务创建时
  | 'TaskCompleted'         // 任务完成时
  | 'ConfigChange'          // 配置变更时
  | 'InstructionsLoaded'    // 指令文件加载时 ← 重要!
  | 'Elicitation'           // 用户选择前
  | 'ElicitationResult'    // 用户选择后
  | 'ToolResult'            // 工具结果处理时
```

7.3 InstructionsLoaded Hook（条件化注入）

这是实现"进入某一路径后注入专属记忆"的关键 Hook：

```
// src/utils/hooks.ts 中定义
interface InstructionsLoadedHookInput {
  path: string            // 被加载的文件路径
  type: MemoryType        // 文件类型: 'User' | 'Project' | 'Local' | 'Managed'
  load_reason: string     // 加载原因: 'session_start' | 'compact' | 'include'
  globs?: string[]        // frontmatter 中的路径模式
  parentFilePath?: string // 通过 @include 引入时的父文件路径
}

// 在 claudemd.ts 中触发
void executeInstructionsLoadedHooks(file.path, file.type, loadReason, {
  globs: file.globs,
  parentFilePath: file.parent,
})
```

7.4 Hook 输出注入

Hook 返回的 JSON 输出通过以下方式注入：

```
// HookResult 中的 systemMessage / additionalContext 字段
interface HookResult {
  systemMessage?: string         // 注入为 system message
  additionalContext?: string    // 追加到上下文中
  blockingError?: string        // 阻止操作
  initialUserMessage?: string   // 替换用户消息
  updatedInput?: object         // 修改工具输入
  // ...
}
```

7.5 asyncRewake 机制（后台 Hook 唤醒）

异步 Hook 可在后台运行，通过 `queued_command` 附件将结果注入到对话中：

```
// src/utils/hooks.ts
void shellCommand.result.then(async result => {
  if (result.code === 2) {
    // exit code 2 = blocking error，注入为 task-notification
    enqueuePendingNotification({
      value: wrapInSystemReminder(`Stop hook blocking error: ${stderr || stdout}`),
      mode: 'task-notification',
    })
  }
})
```

7.6 工作目录信任检查

```
// src/utils/hooks.ts
export function shouldSkipHookDueToTrust(): boolean {
  // 非交互模式（SDK）跳过信任检查
  const isInteractive = !getIsNonInteractiveSession()
  if (!isInteractive) return false
  // 交互模式：所有 Hook 执行前必须接受信任对话框
  const hasTrust = checkHasTrustDialogAccepted()
  return !hasTrust
}
```

---

8. 第七层：System Prompt 分段与动态边界

8.1 SYSTEM_PROMPT_DYNAMIC_BOUNDARY

`src/constants/system.ts` 定义了一个关键边界标记：

```
// src/constants/prompts.ts
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY =
  '__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__'
```

作用：标记 System Prompt 中静态（可跨组织缓存）内容和动态（用户/会话相关）内容的分界线。boundary 之前的部分使用 `scope: 'global'` 缓存，之后的部分则不能。

8.2 缓存断裂注入

`src/context.ts` 中有一个 system prompt 注入机制，用于缓存断裂调试：

```
// src/context.ts
let systemPromptInjection: string | null = null

export function setSystemPromptInjection(value: string | null): void {
  systemPromptInjection = value
  // 清除上下文缓存
  getUserContext.cache.clear?.()
  getSystemContext.cache.clear?.()
}
```

```
// 在 getSystemContext 中:
if (feature('BREAK_CACHE_COMMAND')) {
  injection = getSystemPromptInjection()
  // 注入为 cacheBreaker
  return { cacheBreaker: `[CACHE_BREAKER: ${injection}]` }
}
```

PS：

- cacheBreaker 是一个可控的缓存断裂开关，**仅供内部调试**——通过在每次请求的 system context 中注入一个唯一标识，让工程师能精确控制何时打破 Prompt Cache，借此测量和分析缓存命中行为

8.3 缓存断裂检测

`src/services/api/promptCacheBreakDetection.ts` 监控 prompt 缓存是否被正确使用。

---

9. 第八层：条件化规则注入（Conditional Rules）

9.1 嵌套记忆加载机制

CLAUDE.md 规则支持两种加载策略：

1. **Eager Loading（预加载）**：会话启动时一次性加载所有规则
2. **Nested/Demand Loading（按需加载）**：仅在访问特定文件/目录时才加载条件规则 -> 记忆中frontmatter按 path 进行条件触发那个方法

```
// src/utils/claudemd.ts
// Phase 1: getManagedAndUserConditionalRules - 加载全局条件规则
export async function getManagedAndUserConditionalRules(
  targetPath: string,
  processedPaths: Set<string>,
): Promise<MemoryFileInfo[]>

// Phase 2: getMemoryFilesForNestedDirectory - 加载嵌套目录规则
export async function getMemoryFilesForNestedDirectory(
  dir: string,
  targetPath: string,
  processedPaths: Set<string>,
): Promise<MemoryFileInfo[]>

// Phase 3: getConditionalRulesForCwdLevelDirectory - 加载 CWD 层规则
export async function getConditionalRulesForCwdLevelDirectory(...)
```

9.2 Glob 模式匹配

```
// src/utils/claudemd.ts
// 使用 ignore 库（尊重 .gitignore 语义）
import ignore from 'ignore'
return ignore().add(file.globs).ignores(relativePath)

// 支持的 frontmatter 格式:
// paths:
```

```
//    - "src/**/*.ts"
//    - "src/**/*.tsx"
```

9.3 前置路径解析

路径 glob 相对于不同类型的规则有不同的基准目录：

```
const baseDir =
  type === 'Project'
    ? dirname(dirname(rulesDir)) // .claude/rules/ 的父级 = 项目根
    : getOriginalCwd()           // Managed/User 规则相对于原始 CWD
```

Claude Code 的规则文件（.md）有三个来源，对应三个 type：

type	规则文件目录	典型路径
Managed	~/.claude/rules/	全局配置目录，存放统一管理的规则
User	~/.claude/rules/	用户个人目录
Project	<项目根>/.claude/rules/	随项目代码库一起版本控制的规则

- Project 类型：`rulesDir = /path/to/project/.claude/rules`，向上两级得到 `/path/to/project`——即项目根目录。Project 规则由团队成员在 CLAUDE.md 或 .claude/rules/ 下编写，规则作者描述 glob 时心里的参照系就是"项目根"，所以 `globs: ['src/**/*.ts']` 就是相对于项目根的路径。
- Managed/User 类型：这两个规则放在 `~/.claude/rules/` 下。用户可能在任意目录下运行 claude，规则作者写 glob 时参照的"当前目录"应该是用户启动 claude 时所在的目录（即 getOriginalCwd()），而不是 home 目录。所以 `globs: ['project/src/**/*.ts']` 会被理解为"相对于我运行 claude 的位置"。

---

10. Post-Sampling Hook（采样后注入）

PS：
"采样" / "Sampling" 是 Anthropic API 的术语，指**模型从概率分布中生成一个 token 的过程**。一次完整的 "sample"（采样）就是模型生成一段完整的回复——即一个 assistant 消息，包含所有 content blocks（text、tool_use 等）

Post-Sampling Hook 的时机

```
发送请求 → API 返回流 → 模型完成采样（回复生成完毕）
                              ↓
                        executePostSamplingHooks() ← 在这里执行
                              ↓
                        执行工具（tool_use 触发 CCR）
                              ↓
                        返回结果给模型（继续下一轮）
```

时机： 模型已经完整生成了回复，但还没有开始执行工具
用途： 这类 Hook 的核心价值是利用模型刚生成的内容做后续处理，而不阻塞工具执行链（基于模型输出做副作用/顺便做一些额外的事）

10.1 核心机制

`src/utils/hooks/postSamplingHooks.ts` 定义了一套内部 API，允许在模型采样完成后执行副作用：

```
// Post-sampling hook 注册表（内部 API，不暴露在 settings.json）
const postSamplingHooks: PostSamplingHook[] = []

export function registerPostSamplingHook(hook: PostSamplingHook): void {
  postSamplingHooks.push(hook)
}
```

```
export async function executePostSamplingHooks(
  messages: Message[],
  systemPrompt: SystemPrompt,
  userContext: { [k: string]: string },
  systemContext: { [k: string]: string },
  toolUseContext: ToolUseContext,
  querySource?: QuerySource,
): Promise<void>
```

10.2 使用场景

目前主要用于 Session Memory 的自动提取：

```
// src/services/SessionMemory/sessionMemory.ts
const extractSessionMemory = sequential(async function (context: REPLHookContext) {
  // 仅在主线程运行
  if (context.querySource !== 'repl_main_thread') return
  // 检查 GrowthBook gate（非阻塞，缓存值）
  if (!isSessionMemoryGateEnabled()) return
  // 检查 token 和工具调用阈值
  if (!shouldExtractMemory(messages)) return
  // fork 子代理执行提取
  await runForkedAgent({ querySource: 'session_memory', ... })
})

export function initSessionMemory(): void {
  registerPostSamplingHook(extractSessionMemory)
}
```

现有注册项包括：

Hook	作用
extractSessionMemory	从模型输出中提取值得记忆的内容，写入 auto memory
updateMagicDocs	如果模型改动了 MagicDocs 对应的文件，更新文档缓存
createSkillImprovementHook	分析 skill 执行结果，改进 skill 质量
Prompt 推测 (speculation)	基于模型输出预判下一步可能用到的提示
/auto-dream	生成梦境记忆

与其他 Hook 的区分：

- Pre-Sampling Hook（before-tool-call 等）：工具执行前/后运行，可以修改请求内容或拦截工具
- Post-Sampling Hook：模型回复生成完毕后、工具执行前运行，用于副作用分析
- Stop Hook：模型中途停止时运行，可以注入额外上下文让它继续

---

11. appendSystemPrompt 与 overrideSystemPrompt

11.1 两层额外注入

`src/utils/systemPrompt.ts` 中的 `buildEffectiveSystemPrompt()` 接受两个额外的 prompt 参数：

```
export function buildEffectiveSystemPrompt({
  overrideSystemPrompt,  // 优先级 0: 完全替换所有其他 prompt
```

```
  customSystemPrompt,      // 优先级 3: 自定义 prompt（通过 --system-prompt 指定）
  appendSystemPrompt,      // 始终追加在最后
  defaultSystemPrompt,
  // ...
}): SystemPrompt
```

11.2 优先级体系

```
优先级 0: overrideSystemPrompt → 完全替换，返回 asSystemPrompt([overrideSystemPrompt])

优先级 1: Coordinator 模式 → 使用 coordinator prompt

优先级 2: Agent 模式（mainThreadAgentDefinition）
  - PROACTIVE/KAIROS 模式: agent prompt 追加到 default
  - 其他模式: agent prompt 替换 default

优先级 3: customSystemPrompt（--system-prompt CLI 参数）

优先级 4: defaultSystemPrompt

优先级 5: appendSystemPrompt（始终追加到最后，除 overrideSystemPrompt 外）
```

PS：**KAIROS/PROACTIVE 模式**：

- KAIROS/PROACTIVE 让 Claude Code 从"被叫才应"的工具变成"持续运行、主动汇报"的助手，通过 SendUserMessage 工具和定期 tick 机制实现无需用户驱动的持续工作流
- **持续运行机制**
  - 当 KAIROS 激活且消息队列空闲时，print.ts:2477 会自动注入一个 proactive tick，让模型在没有用户输入的情况下也能"醒来"继续工作——比如检查后台任务状态、推送进度。
- **记忆模式的改变**
  - 在 KAIROS 下，buildAssistantDailyLogPrompt 替换了标准记忆提示：模型不再维护 MEMORY.md 索引，而是追加写入每日日志文件（YYYY/MM/YYYY-MM-DD.md），由一个夜间进程统一整理成 MEMORY.md。这适合长时间运行的会话，避免频繁修改索引文件。

11.3 Cowork 额外指导注入

```
// 通过环境变量注入额外指导
const coworkExtraGuidelines = process.env.CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES
// 会被注入到 Auto Memory 的 buildMemoryLines 中
```

PS：

- Cowork 是 Claude Code 的一个桌面/嵌入式运行模式：**在桌面应用中内嵌运行 Claude Code，而不是通过终端 CLI**。用户可以通过 Cowork 桌面客户端或 Cowork SDK 接入。
- **二者关系**：Cowork 是 Claude Code 的嵌入方/消费者，Claude Code 是底层的引擎层

Cowork 和普通 CLI 的关键区别

- 消息路由由桌面应用控制（Cowork 桥接 WebSocket 而非 REPL）
- 记忆路径由外部控制（通过环境变量注入，而非按 git 根目录自动推断）
- 插件目录用 cowork_plugins 而非 plugins

- 设置文件用 cowork_settings.json 而非 settings.json

CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES 的作用
Cowork 作为嵌入模式，运营方（如企业 IT）需要给模型注入额外的行为指导，这些指导是环境级别的，不写死在代码里。换句话说，CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES 让 Cowork 运营方可以**在环境变量层面给模型的记忆系统注入额外的行为规范，而无需修改 Claude Code 本身的代码**。

例如：企业 Cowork 可能要求模型"所有代码变更必须符合内部代码规范"、"对外通信必须经过特定审批流程"等。这些内容通过这个环境变量注入：

```
const coworkExtraGuidelines = process.env.CLAUDE_COWORK_MEMORY_EXTRA_GUIDELINES
// 注入到 memory prompt 的 extraGuidelines 参数中
```

它被注入到两个地方：

- Auto Memory（loadMemoryPrompt）：普通会话的记忆系统指导
- Agent Memory（agentMemory.ts）：子 Agent 的记忆系统指导

在 buildMemoryLines 中，extraGuidelines 以独立段落追加到记忆说明末尾，模型读取时会直接看到这些额外规则。

CLAUDE_COWORK_MEMORY_PATH_OVERRIDE 的作用
普通 CLI 模式下，记忆目录按 git 根目录哈希推断，每个项目一个目录。但 Cowork 的记忆路径由外部挂载控制（可能是按用户或团队的作用域挂载），所以通过这个环境变量直接覆盖路径计算逻辑，让 Cowork 运营方可以精确控制记忆存储位置。

---

12. Coordinator 与 Agent Prompt 优先级

12.1 Coordinator 模式

PS：**详见 “ Agent算法流程” 相关文档**

当 `feature('COORDINATOR_MODE') && process.env.CLAUDE_CODE_COORDINATOR_MODE` 时，使用 Coordinator Prompt：

```
// src/utils/systemPrompt.ts
if (feature('COORDINATOR_MODE') && isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)) {
  return asSystemPrompt([getCoordinatorSystemPrompt(), ...appendSystemPrompt])
}
```

12.2 Agent 定义注入

内置 Agent（由 `loadAgentsDir.ts` 加载）可以提供每个 Agent 类型专属的 system prompt：

```
// src/tools/AgentTool/loadAgentsDir.ts
const agentSystemPrompt = mainThreadAgentDefinition
  ? mainThreadAgentDefinition.getSystemPrompt()
  : undefined
```

---

13. Compact（上下文窗口压缩）

PS：**详见 “上下文管理” 相关文档**

13.1 压缩触发时机

```
// src/services/compact/autoCompact.ts
// 当上下文接近 token 限制时自动触发压缩
// 触发后通过 summarizeMessages 将旧消息替换为摘要
```

13.2 压缩过程中的记忆处理

```
// src/services/compact/sessionMemoryCompact.ts
// 压缩时 Session Memory 也参与摘要过程
// extractMemories 服务在压缩后重新加载记忆（resetGetMemoryFilesCache('compact')）
```

13.3 compact boundary 标记

```
// src/utils/messages.ts
export function createCompactBoundaryMessage(
  partialCompactDirection?: PartialCompactDirection
): SystemCompactBoundaryMessage
// 在压缩后的消息序列中插入边界标记，便于后续追踪
```

---

14. 架构总结与关键设计模式

14.1 分层职责一览

层	机制	触发时机	持久化
System Reminder	`<system-reminder>` XML 标签	始终（通过消息附件）	否
CLAUDE.md	文件加载	会话启动 + 按需	否（文件本身持久）
Auto Memory	引导 + 记忆文件	会话启动 + 提取时	是（磁盘文件）
Session Memory	Fork 子代理	Post-Sampling	是（会话级磁盘文件）
ExtractMemories	Fork 子代理	StopHooks	是（Auto Memory）
Hooks	Shell/回调	生命周期事件	视 Hook 而定
Coordinator/Agent	Prompt 替换/追加	CLI flag / feature	否
Compact	摘要替换	Token 阈值	否

14.2 关键设计模式

模式 1：Forked Agent（隔离执行）

所有可能产生副作用的后台操作（Session Memory、ExtractMemories）都通过 `runForkedAgent()` 创建隔离上下文执行，不污染主对话状态：

```
// src/utils/forkedAgent.ts
await runForkedAgent({
  promptMessages,
  cacheSafeParams,         // 继承父会话的 prompt cache
  canUseTool,               // 限制工具权限
  overrides: { readFileState: isolatedContext.readFileState }
})
```

模式 2：GrowthBook Feature Gate

几乎所有新机制都有 GrowthBook feature flag 控制，实现灰度发布：

```
if (!getFeatureValue_CACHED_MAY_BE_STALE('tengu_passport_quail', false)) return
if (!getFeatureValue_CACHED_MAY_BE_STALE('tengu_session_memory', false)) return
```

PS：

- 定义：GrowthBook 是一个 A/B 测试 / 功能开关平台，Claude Code 用它来灰度发布功能和远程控制代码行为，无需重新部署
- 两种使用方式
  - 构建时常量折叠（feature('FLAG_NAME')）：这类 flag 用于大的产品线切换（KAIROS、BRIDGE_MODE、PROACTIVE 等），在打包时决定是否包含某段代码
  - 运行时远程评估（getFeatureValue_CACHED_MAY_BE_STALE）：这类 flag 的值从 GrowthBook 服务器获取，可以随时修改，用于：1）按用户/组织灰度开启某个小功能；2）实时关闭某个线上 feature（kill-switch）；3）A/B 测试不同参数值（如阈值、超时时间）
- **用户属性定向**：GrowthBook 根据用户 attributes（用户类型、订阅等级、组织 ID 等）决定每个 flag 的返回值，实现分用户群体控制

模式 3：Memoize 缓存

大量上下文数据通过 `lodash-es/memoize` 缓存，避免重复计算：

```
export const getSystemContext = memoize(async () => { ... })
export const getUserContext = memoize(async () => { ... })
export const getMemoryFiles = memoize(async () => { ... })
```

模式 4：缓存清除传播

当影响上下文的值改变时，级联清除所有相关缓存：

```
export function setSystemPromptInjection(value: string | null): void {
  systemPromptInjection = value
  getUserContext.cache.clear?.()
  getSystemContext.cache.clear?.()
}
```

模式 5：System Prompt 分段缓存

`systemPromptSection()` 支持 `cacheBreak: boolean` 标志，允许某些段每轮重新计算而不破坏整个 prompt cache：

```
// 稳定段：cacheBreak: false（可缓存）
export function systemPromptSection(name: string, compute: ComputeFn): SystemPromptSection

// 动态段：cacheBreak: true（每轮重算，会破坏 prompt cache）
export function DANGEROUS_uncachedSystemPromptSection(
  name: string,
  compute: ComputeFn,
  _reason: string,  // 必须提供原因说明
): SystemPromptSection
```

模式 6：时机感知的 Extract Mem

ExtractMemories 在主 Agent **不产生**记忆写入时才工作（互斥），避免与主 Agent 竞争写入同一文件。

模式 7：Memory Type 分类约束

通过 frontmatter `type` 字段强制分类，附带详细的 save/access/trust 指导，防止记忆系统被滥用为通用存储。

附录：关键文件索引

文件	职责
`src/utils/messages.ts`	System Reminder 包装、消息归一化、compact boundary
`src/context.ts`	getSystemContext / getUserContext（git status, CLAUDE.md, date）
`src/utils/claudemd.ts`	完整 CLAUDE.md 发现、加载、解析、缓存管道
`src/memdir/memdir.ts`	Auto Memory 提示构建、入口截断
`src/memdir/memoryTypes.ts`	记忆类型分类、save/access/trust 指导文本
`src/services/SessionMemory/sessionMemory.ts`	会话记忆自动提取（Post-Sampling）
`src/services/extractMemories/extractMemories.ts`	对话历史自动提取为持久记忆（StopHooks）
`src/utils/hooks.ts`	完整 Hook 系统（16+ 事件类型）
`src/utils/hooks/postSamplingHooks.ts`	采样后 Hook 机制
`src/constants/prompts.ts`	System Prompt 各节构建
`src/constants/systemPromptSections.ts`	分段缓存管理
`src/utils/systemPrompt.ts`	有效 System Prompt 优先级构建
`src/utils/queryContext.ts`	API cache key 前缀构建
`src/services/compact/compact.ts`	上下文压缩
`src/services/compact/autoCompact.ts`	自动压缩触发
`src/utils/dynamicConfig.ts` / `src/hooks/useDynamicConfig.ts`	动态配置（GrowthBook）